Python Team Project Report

Baron Chiang, Tanmay Deshmukh, Leo Cohen

LC5: Team 25

ENGR 13300: Transforming Ideas to Innovation For EPICS/VIP

Professor Aamod Samuel

10/19/2026

1. Project Motivation

Write your project motivation section here. Discuss how image processing and classification techniques are used in your chosen engineering discipline, citing at least three appropriate references.

2. Project Overview and Methods

Write your project overview and methods section here. This includes a description of algorithms, a summary of background research, and evidence to support the methods chosen for each of the processing steps.

Checkpoint 1 Task 1

For Task 1 For The Checkpoint 1, the main purpose of the script is to convert an image from a RGB format to a HSV format for an image. This is important for our project as HSV separates the color of a pixel from the intensity of the pixel, while RGB has all of the color and saturation information mixed into the color information. This makes it easier for a computer to understand the information from the pixel and makes it easier for object detection algorithms. We would do this through a few different functions: load_img, resize_image, pad_image, rgb_to_hsv, and convert_to_hsv. The load_img function will load an image from a file directory, the resize_image function will resize the image to a specified size, the pad_image will add pixels to the image so the image doesn't stretch, the rgb_to_hsv function will convert the RGB values of a single pixel to HSV, and the convert_to_hsv function will convert a whole image from RGB to HSV. For each of the methods chosen here, they were verified against the autograder and the sample outputs within the ENGR 13300 website.

Checkpoint 1 Task 2:

Task 2 implements spatial filtering by sliding a kernel across a grayscale image. The function create_mean_kernel() builds an $n \times n$ mean (box) kernel in which every element equals $1/n^2$, so the elements sum to 1 and overall brightness is preserved. Following the course specification, pad_image() uses Pillow's

ImageOps.pad (bilinear resampling) to extend the image to (W+w−1)×(H+h−1), so the filtered output keeps the original dimensions. apply_kernel() then moves the kernel across every pixel in a nested loop, multiplies the covered region element-wise by the kernel, and sums the products to produce each output value. Averaging neighboring pixels suppresses noise, so larger kernels blur more. main() verifies the method on a 5×5 test image with one bright pixel, whose intensity should spread evenly across its neighborhood.

Checkpoint 1 Task 3

This project implements a image processing system using Python and Numpy to preform a Gaussian filter for noise reduction and preserve features. The workflow begins by loading and converting user-selected RGB images into normalized 2D grayscale arrays. To process the images, a normalized 2D Gaussian kernels constructed based on a user defined sigma which acts as the standard deviation to ensure proper edge weighting defined by kernel_size = 2 x [2sigma] + 1. The filtering stage applies zero-padding to handle image boundaries before sliding the kernel across the array with a stride of 1 to compute weight local sums. Compared to standard mean filtering, which assigns equal weight and smoothing over lots of different features.

Checkpoint 2 Task 1

This task creates a Sobel filter then uses it to create images that are used for edge detection. This is used for machine learning as it is easier to determine what an image is without the color and saturation information and just where the edges are. The script works by asking the user for a path to an image file, then the image gets converted into an array, gets turned to gray scale, then blurred, then put through the sobel filter. The final image is then displayed and saved into the script folder. The functions to get the image from the directory, turning to grayscale and blurring all come from previous tasks while the sobel filter is a new one created just for this task.

Checkpoint 2 Task 2

Task 2 turns each traffic sign image into a short list of numbers that a computer can use to tell a Deer Warning sign from a Left Turn Ahead sign. Each image is first resized to 100×100 pixels so they are all the same size. For the color features, every pixel is converted from red-green-blue into hue, saturation and value. Hue is the basic color, saturation is how vivid it is, and value is how bright it is. We then take the average and the spread (standard deviation) of each across the whole image. This gives six numbers that describe the sign's overall color. For the circle check, the image is turned to grayscale and blurred to remove noise. Then a circle-finding tool (the Hough circle transform from CV2) looks for round shapes of a reasonable size. If it finds one, the image gets a 1, otherwise a 0. For triangles, we blur the image, trace its edges with a Sobel filter, and then use OpenCV's findContours to follow each outline. Next, approxPolyDP simplifies each outline down to its corners. If an outline has exactly three corners and is big enough not to be noise, the image gets a 1 for has_triangle, otherwise a 0. We also count the straight lines using the Sobel edge transform followed by HoughLinesP function from CV2, using the same edge picture, because the Left Turn arrow should have more of them than the deer sign. We chose these methods because they are common, well-tested ways to measure color and find shapes.

Checkpoint 2 Task 3

Task 3 works to scale the feature data extracted from Checkpoint 2 Task 2 that includes: hue_mean, hue_std saturation_mean, saturation_std, value_mean, value_std, number_lines, has_circle, has_triangle. The program applies feature scaling to get each feature into a common range. The program then plots the features on ether a 3 axis or 2 axis graph to be able to visualize clusters of photographs that are grouped together. This is useful for the project overall because it is a way to both mathematically and visually sort important features in any photograph like if they have circles, triangles, or what color values are present.

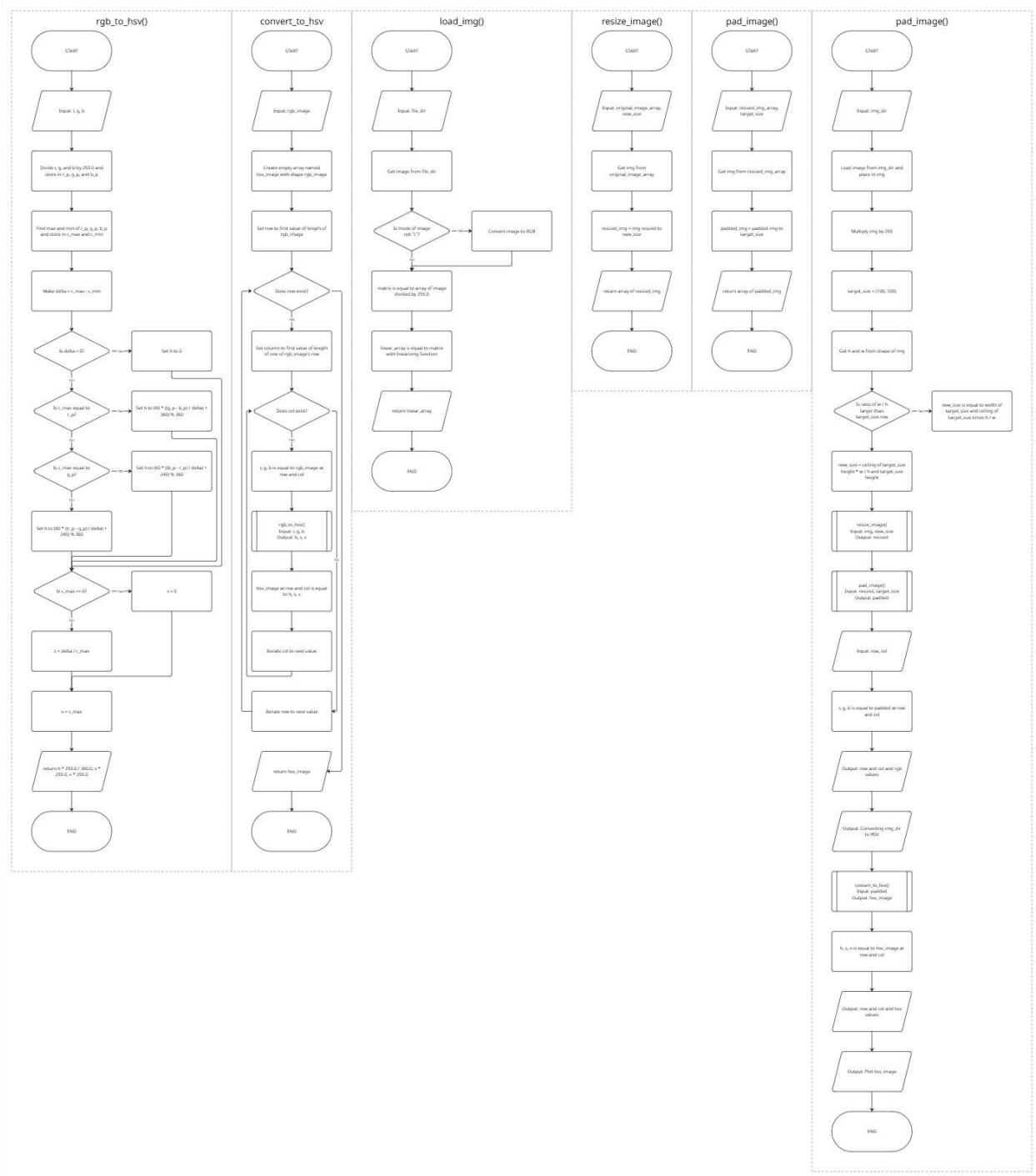
3. Discussion of Algorithm Design

Write a detailed discussion of your algorithm here. Be sure to include properly labeled flowcharts of the overall program and user-defined functions and design rationale for modularizing code.

Checkpoint 1 Task 1

The first function that we created was the rgb_to_hsv function. This function had RGB values from 0 to 255 as the input and HSV values from 0 to 255 as the output. This function is used to convert a single pixel's values from RGB to HSV. We did this by using the equations 7.1 to 7.6 to convert. The next function, convert_to_hsv, uses a nested for loop to loop through each pixel in the input image and converts the pixels to HSV then outputs a HSV image. The last three functions, load_img, resize_img, and pad_image all came from Python 5 Team Task 3 and allow us to load, resize, then pad the image so that we would have a standard sizing format for our image.

For the main function, we ask the user to input an image file directory, then we find the image from that directory. We then convert it to 255 and as an int8. The if statement within the main is to make sure that we aren't losing any precision when we pad and resize the image and that the final image meets the target image width and height. We made the two different functions to make our main function smaller, as we use convert_to_hsv when we are displaying the converted HSV image and the rgb_to_hsv for a single pixel coordinate. We made all of these different functions to reduce the amount of lines within the script and reduce complexity as well. By creating separate functions, we were able to check for errors within a single function instead of trying to check for errors in one large script.


rgb_to_hsv()
START
Input: r, g, b
Divide r, g, and b by 255.0 and store in r_p, g_p, and b_p
Find max and min of r_p, g_p, b_p and store in c_max and c_min
Make delta = c_max - c_min
Is delta = 0?
yes
Set h to 0
no
Is c_max equal to r_p?
yes
Set h to (60 * ((g_p - b_p) / delta) + 360) % 360
no
Is c_max equal to g_p?
yes
Set h to (60 * ((b_p - r_p) / delta) + 240) % 360
no
Set h to (60 * ((r_p - g_p) / delta) + 240) % 360
Is c_max == 0?
yes
s = 0
no
s = delta / c_max
v = c_max
return h * 255.0 / 360.0, s * 255.0, v * 255.0
END
convert_to_hsv
START
Input: rgb_image
Create empty array named hsv_image with shape rgb_image
Set row to first value of length of rgb_image
Does row exist?
yes
Set column to first value of length of one of rgb_image's row
Does col exist?
yes
r, g, b is equal to rgb_image at row and col
rgb_to_hsv()
Input: r, g, b
Output: h, s, v
hsv_image at row and col is equal to h, s, v
Iterate col to next value
no
Iterate row to next value
no
return hsv_image
END
load_img()
START
Input: file_dir
Get image from file_dir
Is mode of image not "L"?
yes
Convert image to RGB
no
matrix is equal to array of image divided by 255.0
linear_array is equal to matrix with linearizing function
return linear_array
END
resize_image()
START
Input: original_image_array, new_size
Get img from original_image_array
resized_img = img resized to new_size
return array of resized_img
END
pad_image()
START
Input: resized_img_array, target_size
Get img from resized_img_array
padded_img = padded img to target_size
return array of padded_img
END
pad_image()
START
Input: img_dir
Load image from img_dir and place in img
Multiply img by 255
target_size = (100, 100)
Get h and w from shape of img
Is ratio of w / h larger than target_size row
yes
new_size is equal to width of target_size and ceiling of target_size times h / w
new_size = ceiling of target_size height * w / h and target_size height
resize_image()
Input: img, new_size
Output: resized
pad_image()
Input: resized, target_size
Output: padded
Input: row, col
r, g, b is equal to padded at row and col
Output: row and col and rgb values
Output: Converting img_dir to HSV
convert_to_hsv()
Input: padded
Output: hsv_image
h, s, v is equal to hsv_image at row and col
Output: row and col and hsv values
Output: Plot hsv_image
END

Checkpoint 1 Task 2

The program runs in three stages, each handled by a separate user-defined function, with main() coordinating them. main() builds a 5×5 uint8 test image that is zero everywhere except for a single 255 pixel at the center. It then prompts the user for an integer kernel size n and passes n to create_mean_kernel(). That function computes the weight 1/(n·n) and uses np.full to fill an n×n array with it, so the kernel always sums to 1 and filtering preserves average brightness. main() then passes the test image and kernel to apply_kernel(). apply_kernel() reads the image dimensions (H, W) and kernel dimensions (h, w), then initializes an H×W output array R of zeros. It calls pad_image() to enlarge the image to (W + w − 1) × (H + h − 1). Inside pad_image(), the array is converted to a Pillow image and ImageOps.pad extends it with bilinear resampling and a fill of zeros. Because the black border contributes zero to every weighted sum, pixels near the edge are averaged without introducing artificial brightness. The result is converted back to a NumPy array. Back in apply_kernel(), a nested loop visits each output position (j, i). For each position it slices the h×w region of the padded array starting at [j, i], multiplies that region element-wise by the kernel, and stores the sum in R[j, i]. Because the padded image is larger than the original by exactly w − 1 columns and h − 1 rows, every slice is a full h×w window and the loop never indexes out of bounds. The loop produces a filtered array of the same size as the input. Finally, main() prints the kernel size, the kernel, and the filtered image. Using the 5×5 impulse image as the test case makes the result easy to check. A correct mean filter spreads the single bright pixel into an n×n block of equal values, each equal to $255/n^2$, so any error in the kernel or the sliding logic shows up immediately.

Splitting the program into create_mean_kernel(), pad_image() and apply_kernel() gives each function one job, which makes the code easier to write, test and debug. Each function can be tested alone. For example, create_mean_kernel(3) should return nine entries of 1/9 that sum to 1, without

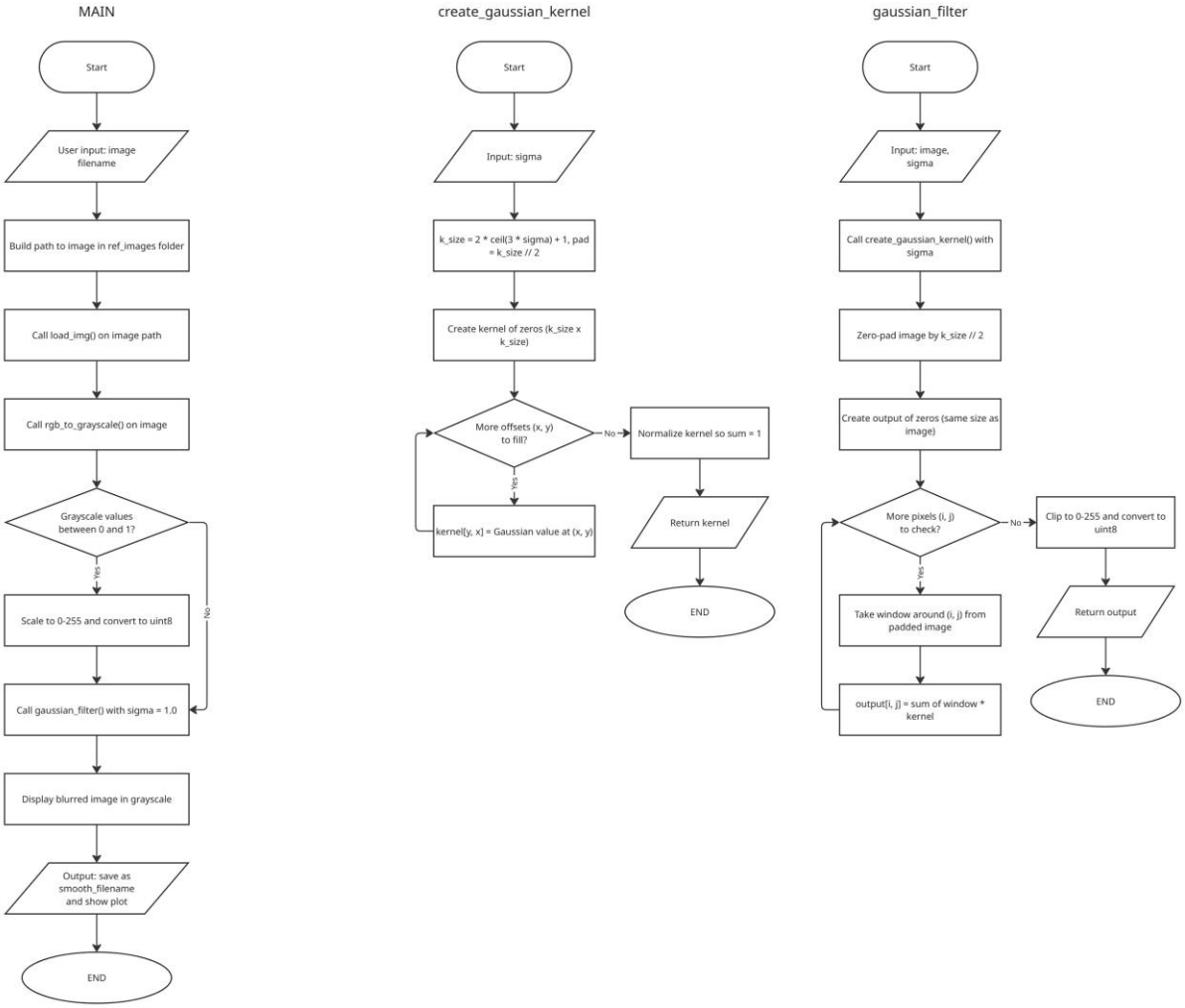
needing any image. pad_image() isolates the padding step, which the course specifies and which may change independently of the filtering math. Keeping it separate means the padding method could be replaced without touching the sliding loop. It also keeps apply_kernel() short enough to read as the core algorithm. Separating the kernel from the filtering also makes the code reusable. A different kernel, such as a Gaussian or edge-detection kernel, could be passed to apply_kernel() unchanged, since that function only assumes a 2-D array of weights. Finally, putting input and printing in main() keeps the computation functions free of user interaction. This lets teammates work on separate functions in parallel and combine them through a clear interface of arrays in and arrays out.


main
Start
User Input For Kernel Size
Create mean kernel using UDF and kernel size from user
make (k * k) float array with all values equal to 1/(k * k)
Sum of 1, Values Equal
Build 5x5 demo image with center pixel 255, others 0
apply_kernel(demo image, kernel)
Print Kernel Size, Kernel Sum, Kernel Array and Filtered Image Array
END
apply_kernel
Start
pad_image(image, target_size)
(Old function)
Create blank output image, same size as the original
Move to first pixel and center kernel on this pixel
Multiply kernel by pixels beneath it
Add up the products and save as output pixel
More pixels?
YES
Move one pixel over
NO
Return Output Image
END

Checkpoint 1 Task 3

We started off by making the create_gaussian_kernel function. Basically, you give it sigma (the standard deviation), and it spits out a 2D Gaussian kernel as a NumPy array. To figure out the size, we use 2 times the ceiling of 3 times sigma, plus 1. That way, the kernel is always odd-sized and big enough to cover almost the whole Gaussian curve. We loop through each spot in the kernel, calculate its value with the Gaussian formula, and then normalize everything so the values add up to 1. That way, the image doesn't get brighter or darker after blurring. Next up is gaussian_filter. This one takes a grayscale image and sigma, builds the kernel, zero-pads the image so the edges work out, and then slides the kernel over every pixel. For each pixel, it multiplies the window by the kernel, adds it up, and that's the new pixel value. We clip the output to 0–255 and make sure it's an 8-bit unsigned integer. The last two functions, load_img and rgb_to_grayscale, just help us load the image and turn it into grayscale using a weighted sum of the red, green, and blue channels. That way, the filter only has to deal with one channel.

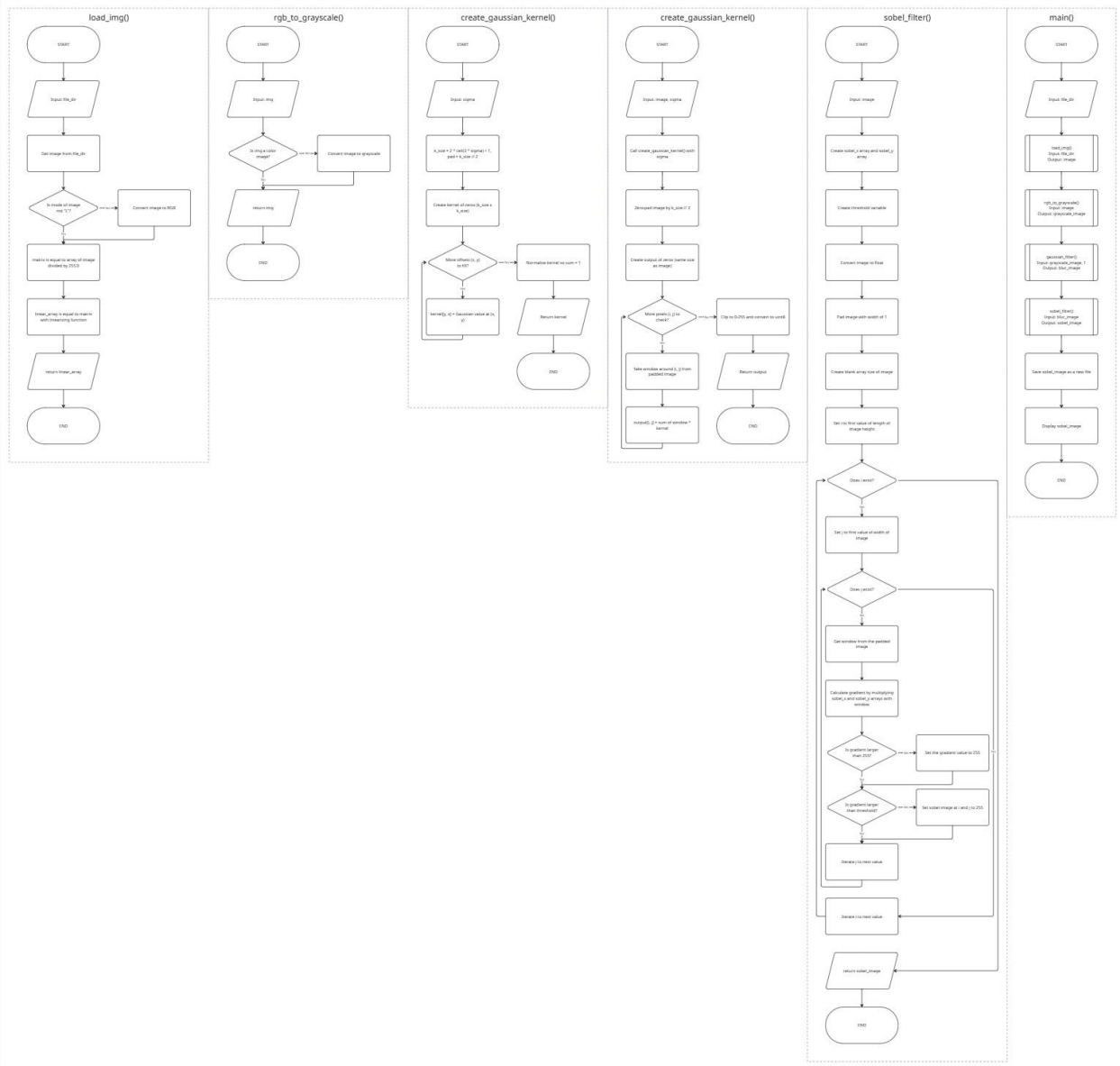
In the main function, we ask for the image file name and look for it in the ref_images folder (same place as the script). After that, we turn the image into grayscale and make sure it's scaled from 0 to 255 as a uint8. There's an if statement to double-check the image is on the right scale, since sometimes images load from 0 to 1 instead. Then we set sigma to 1.0, run the Gaussian filter, show the blurred image, and save it with "smooth_" at the front of the file name. We split up the kernel creation and the filtering into two functions so the main function stays short, and we can reuse create_gaussian_kernel with any sigma. Making everything into separate functions also keeps the script less cluttered and easier to check for mistakes—way better than hunting for bugs in one giant block of code.


MAIN
Start
User input: image filename
Build path to image in ref_images folder
Call load_img() on image path
Call rgb_to_grayscale() on image
Grayscale values between 0 and 1?
Yes
No
Scale to 0-255 and convert to uint8
Call gaussian_filter() with sigma = 1.0
Display blurred image in grayscale
Output: save as smooth_filename and show plot
END
create_gaussian_kernel
Start
Input: sigma
k_size = 2 * ceil(3 * sigma) + 1, pad = k_size // 2
Create kernel of zeros (k_size x k_size)
More offsets (x, y) to fill?
No
Yes
kernel[y, x] = Gaussian value at (x, y)
Normalize kernel so sum = 1
Return kernel
END
gaussian_filter
Start
Input: image, sigma
Call create_gaussian_kernel() with sigma
Zero-pad image by k_size // 2
Create output of zeros (same size as image)
More pixels (i, j) to check?
No
Yes
Take window around (i, j) from padded image
output[i, j] = sum of window * kernel
Clip to 0-255 and convert to uint8
Return output
END

Checkpoint 2 Task 1

This tasks uses a few different functions: load_img, rgb_to_grayscale, create_gaussian_kernel, gaussian_filter, and sobel_filter. All of the user defined functions except for sobel_filter were created in previous tasks. The sobel_filter was a defined function on its own as it allows for new scripts to be able to use the function for edge detection. The sobel_filter function starts by creating 2 3x3 arrays, one for the sobel x filter and the other for the sobel y filter. After this step is done, we then place the threshold value at 50.0, as was stated in the task assignment. Then we create an array of floats from the image, the pad the image with a width of 1 around, then we create a new array the size of the original image to place the new values into. We then initialize two different variables img_h and img_w from the dimensions of the image. We then loop through each pixel in the image dimensions using a nested for loop with the img_h and img_w as the limits. We create a new variable window which is a section of the image as a 3x3 matrix around the pixel, then we calculate the gradient as the square root of the dot product of the window and sobel_x to the power of 2 plus the dot product of the window and sobel_y to the power of 2. We then check if the gradient value is above 255. If it is, then we make it equal to 255. Then we check if the gradient is above the threshold, if it is, then the sobel_image array at the pixel coordinate is equal to 255. After looping through each pixel, we then return the sobel_image array with all values converted to integers.

Keeping the functions separate allows someone to be able to see the sobel_image filter and import the code into their script for use. It also make it easier to code the task 1 script, as we know that all of the other functions except for sobel filter are correct and working, which means that if there was an error with the script, we would focus on the sobel filter function for any errors first.


load_img()
START
Input: file_dir
Get image from file_dir
Is mode of image not 'L'?
no
Convert image to RGB
yes
matrix is equal to array of image divided by 255.0
linear_array is equal to matrix with linearizing function
return linear_array
END
rgb_to_grayscale()
START
Input: img
Is img a color image?
yes
Convert image to grayscale
no
return img
END
create_gaussian_kernel()
START
Input: sigma
k_size = 2 * ceil(3 * sigma) + 1, pad = k_size // 2
Create kernel of zeros (k_size x k_size)
More offsets (x, y) to fill?
yes
kernel[y, x] = Gaussian value at (x, y)
no
Normalize kernel so sum = 1
Return kernel
END
create_gaussian_kernel()
START
Input: image, sigma
Call create_gaussian_kernel() with sigma
Zero-pad image by k_size // 2
Create output of zeros (same size as image)
More pixels (i, j) to check?
yes
Take window around (i, j) from padded image
output[i, j] = sum of window * kernel
no
Clip to 0-255 and convert to uint8
Return output
END
sobel_filter()
START
Input: image
Create sobel_x array and sobel_y array
Create threshold variable
Convert image to float
Pad image with width of 1
Create blank array size of image
Set i to first value of length of image height
Does i exist?
yes
Set j to first value of width of image
Does j exist?
yes
Get window from the padded image
Calculate gradient by multiplying sobel_x and sobel_y arrays with window
Is gradient larger than 255?
yes
Set the gradient value to 255
no
Is gradient larger than threshold?
yes
Set sobel image at i and j to 255
no
Iterate j to next value
Iterate i to next value
return sobel_image
END
main()
START
Input: file_dir
load_img()
Input: file_dir
Output: image
rgb_to_grayscale()
Input: image
Output: grayscale_image
gaussian_filter()
Input: grayscale_image, 1
Output: blur_image
sobel_filter()
Input: blur_image
Output: sobel_image
Save sobel_image as a new file
Display sobel_image
END

Checkpoint 2 Task 2

This program builds a feature table for traffic sign images. For each image, it measures color and checks for lines, circles and triangles. It then saves the results to a CSV file. The functions load_img, rgb_to_hsv, convert_to_hsv, rgb_to_grayscale, resize_image, pad_image and the Gaussian filter (create_gaussian_kernel and gaussian_filter) were created in previous tasks. The sobel_filter was taken from Task 1 of this checkpoint. The functions below are new in this task.

detect_circle(gray_img)

This function checks if the image has a circle. It first blurs the grayscale image with the Gaussian filter. Blurring removes small noise that could look like a circle. Then it uses OpenCV's Hough circle tool (cv2.HoughCircles) to look for round shapes with a radius between 20 and 50 pixels. That size fits a sign inside a 100×100 image. It returns 1 if any circle is found and 0 if not.

detect_triangle(gray_img)

This function checks if the image has a triangle. It blurs the image and finds the edges with the Sobel filter. Then cv2.findContours follows the edges and collects each outline. For each outline, cv2.approxPolyDP simplifies it down to its main corners. If an outline has exactly 3 corners and its area is over 625 pixels, it counts as a triangle. The area rule ignores tiny shapes caused by noise. It returns 1 if a triangle is found and 0 if not.

count_lines(gray_img)

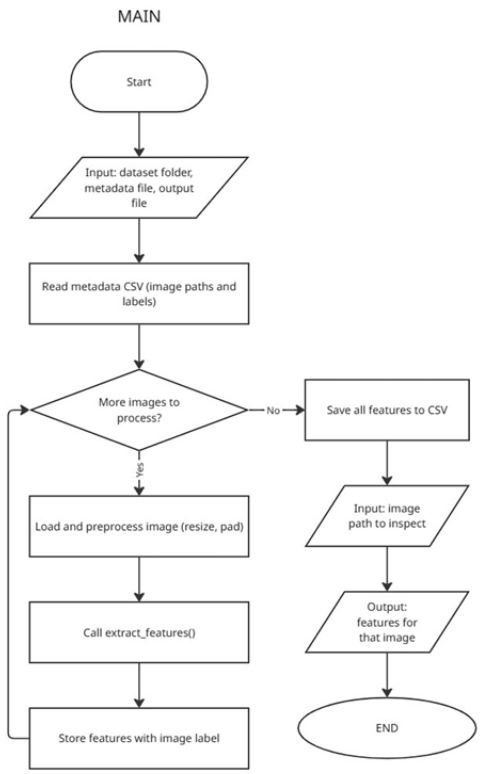
This function counts the straight lines in the image. It first finds the edges with the Sobel filter. Then it uses OpenCV's Hough line tool (cv2.HoughLinesP), which finds straight line pieces in the edges. A line must be at least 30 pixels long, and small gaps of up to 10 pixels are allowed. It returns the number of lines found, or 0 if there are none.

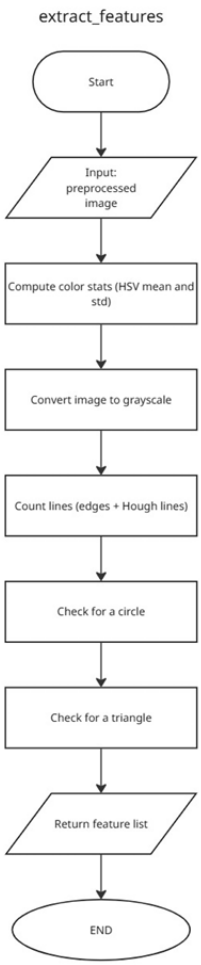
extract_features(img_array)

This function turns one image into a list of 9 numbers. It converts the image to HSV (hue, saturation, value) and takes the mean and standard deviation of each. That gives six color numbers. It then converts the image to grayscale and calls count_lines, detect_circle and detect_triangle to get the last three.

main()

This function runs the whole process. It asks the user for the dataset folder, the metadata file and the output file name. For each image listed in the metadata, it loads the image, resizes it to 100×100, pads it to keep its shape, and calls extract_features. All rows are put in a table with the image path and class ID, and the table is saved as a CSV file. It then prints the table's shape and first rows. Last, it asks for an image path and prints that image's features.



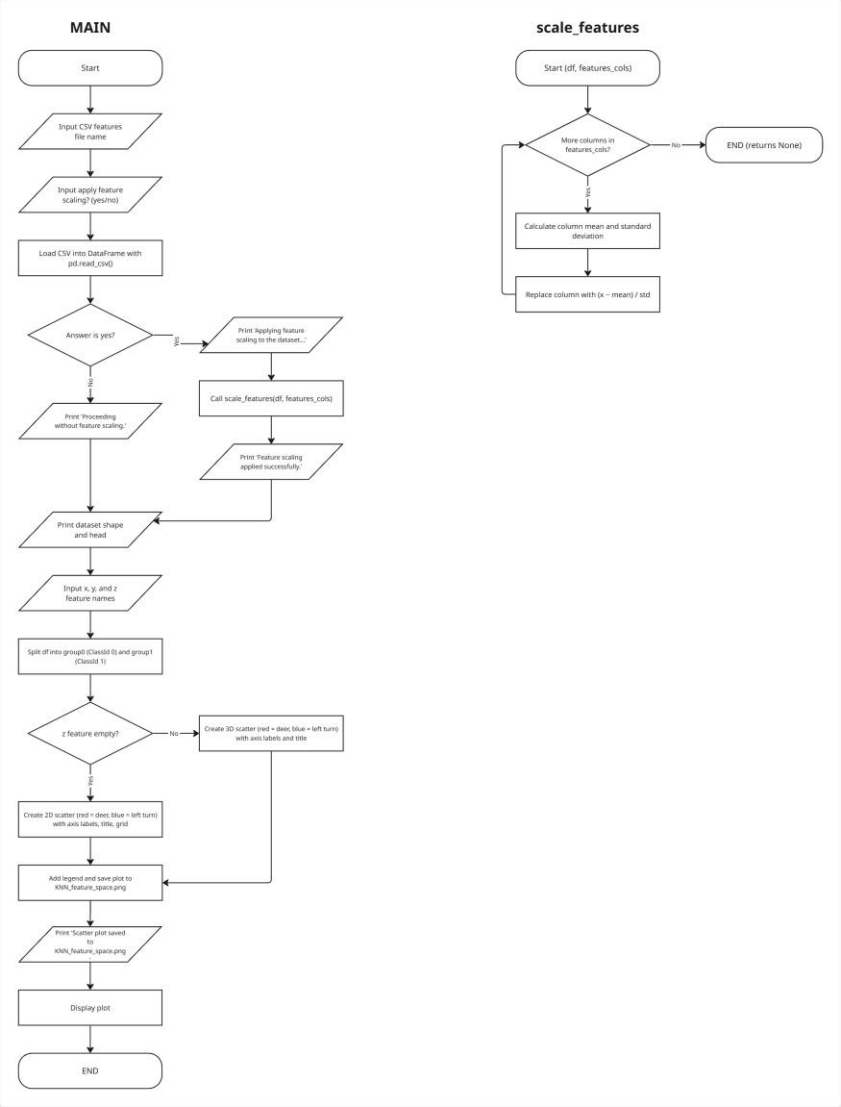


Checkpoint 2 Task 3

The program starts in main where it asks the user for the name of the image features CSV file (the one produced in Task 2) and whether or not to apply feature scaling. The CSV is loaded into a pandas DataFrame using pd.read_csv(). If the user says "yes" to scaling then main prints a status message, calls the user supplied function scale_features with the DataFrame and a hard coded list of the 9 feature columns and then prints a confirmation when scaling is complete. If the user says "no" then it simply prints that it is proceeding without scaling. It then prints the shape of the dataset and the first 5 rows to confirm the result. It then asks for the x, y and (optionally) z axis features. The DataFrame is split into two groups by ClassId (0 is deer warning signs, 1 is left turn ahead signs) and each group is plotted in its own colour (red for deer warning, blue for left turn ahead), both with alpha=0.6 to make them semi-transparent. A decision as to whether or not the z-axis input is empty determines whether a 2D scatter plot with a grid is produced or a 3D scatter plot. In either case the plot has axis labels, a title and a legend and is saved as KNN_feature_space.png prior to a confirmation message being printed. scale_features takes each column in the feature list in turn, calculates the mean and standard deviation of that column and then replaces each value with $(x - \mu)/\sigma$. It modifies the DataFrame in place and returns None. The flowcharts for main and scale_features are shown in the flow chart.

We've made feature scaling its own function because it's a completely separate, standalone operation which doesn't require any user input or plotting. It can be tested on its own (and the autograder will do this) and is something that we'll want to re-use in later tasks (e.g. the KNN classifier) where we'll need to apply the same scaling to new data. Modifying the DataFrame in place rather than creating a second copy of the 1080 row dataset saves memory. It also means that the main function is only responsible for the things that it needs to be: handling the input and output, deciding whether or not to scale and building the plot. The functions are all kept very short and focused which makes the

code much easier to read and debug and allows one person to change the plotting code without affecting the scaling code (or vice versa).


MAIN
Start
Input CSV features file name
Input apply feature scaling? (yes/no)
Load CSV into DataFrame with pd.read_csv()
Answer is yes?
Yes
Print 'Applying feature scaling to the dataset...'
Call scale_features(df, features_cols)
Print 'Feature scaling applied successfully.'
No
Print 'Proceeding without feature scaling.'
Print dataset shape and head
Input x, y, and z feature names
Split df into group0 (ClassId 0) and group1 (ClassId 1)
z feature empty?
No
Create 3D scatter (red = deer, blue = left turn) with axis labels and title
Yes
Create 2D scatter (red = deer, blue = left turn) with axis labels, title, grid
Add legend and save plot to KNN_feature_space.png
Print 'Scatter plot saved to KNN_feature_space.png'
Display plot
END
scale_features
Start (df, features_cols)
More columns in features_cols?
No
END (returns None)
Yes
Calculate column mean and standard deviation
Replace column with (x − mean) / std

References

List your references here using the MLA format. See Purdue OWL for help with formatting.

van Rossum, G. (2026, October 1). Python 3.13 documentation. Python documentation. https://docs.python.org/3.13/

The scikit-image contributors. (2023). Image filtering. The scikit-image. https://scikit-image.org/skimage-tutorials/lectures/1_image_filters.html

4. Appendix

1. **User manual**

   This should be a guide for operating the program along with sample inputs and outputs (including pictures).

2. **Project management plan** *(*Team members' contribution*)*

   Summary of the contributions of each member of the team to the project. Describe your team's methods for collaboration that facilitated active participation and learning by all team members. Include a discussion of opportunities for improvement for future team projects.

3. **Discussion of design process** *(Approach to the design process)*

   Description of how your team followed the design process, which is defined as "a process of devising a system, component, or process, to meet desired needs within constraints. It is an iterative, creative, decision-making process that involves developing requirements, performing analysis and synthesis, generating multiple solutions, evaluating solutions against requirements, considering risks, and making trade-offs".

4. **Code**

The team should upload the Python files, but they should also include the code as an appendix in the report. Code should be well commented to increase readability for future users of the algorithm.